



C9

COUNCIL OF NINE

Candidate Intelligence

The AI Recruiting Brain

An explainable engine that reads profiles instead of counting keywords — ranking 100,000 candidates in ~2 minutes on CPU, fully offline.

Team: Rainlight

Team Leader: Varshith P

Track 1 — The Data & AI Challenge · Intelligent Candidate Discovery & Ranking

India Runs by Redrob AI · Idea Submission

Solution Overview

What is our proposed solution?

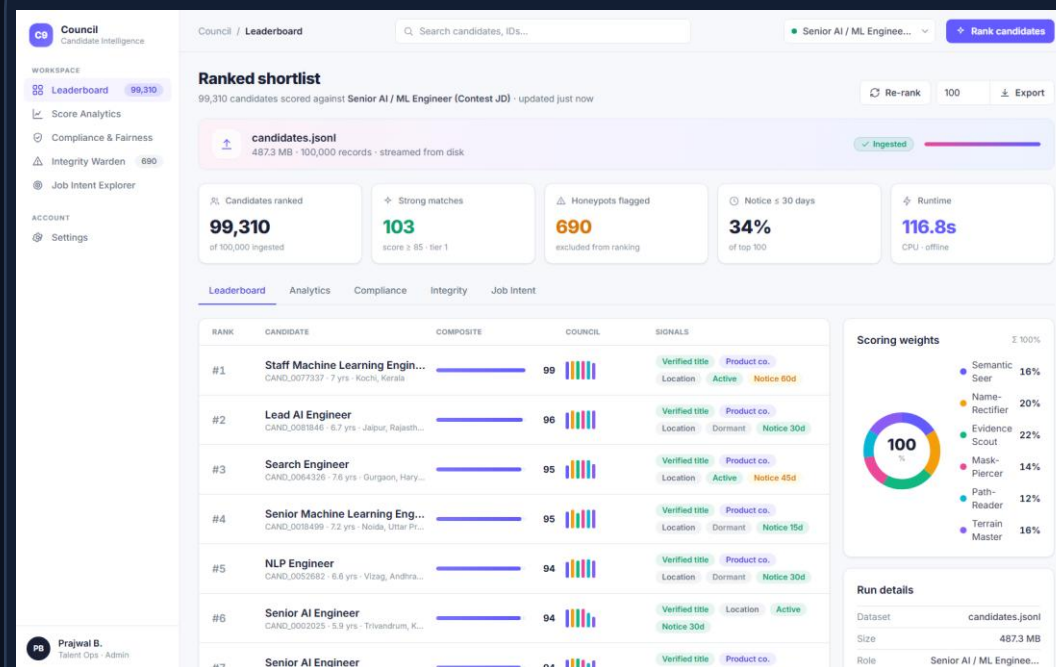
- The “Council of Nine” — a candidate-ranking engine where nine interpretable sub-scorers each judge a different dimension of fit, then fuse into one calibrated score.
- It reads the career narrative (titles, role descriptions, tenure, company type) far above the self-declared skills list.
- Ships an end-to-end product: drag-drop dataset, role selector, tunable weights, ranked leaderboard, per-candidate reasoning, Excel export, and a live activity log.

What differentiates us from traditional matching?

Evidence > assertion. We discount skills that aren't backed by endorsements or real usage — neutralising keyword-stuffers.

Trap-resistant. An Integrity Warden floors logically impossible “honeypot” profiles.

Explainable & fast. Every rank carries a grounded reason; runs CPU-only, offline.



JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- Production embeddings / retrieval, vector DBs & hybrid search
- Demonstrated ranking / search / recommendation systems at product companies
- 5–9 yrs experience (6–8 ideal); strong Python; ranking-eval literacy (NDCG/MRR/MAP)

Hard dis-qualifiers: services-only careers, title-chasers, <12-mo LangChain-only, CV/speech-only without NLP/IR.

Signals that determine relevance (beyond keywords)

Role identity — current title must match reality (an “HR Manager” with AI skills is penalised).

Career evidence — mining role descriptions for “built / shipped / scaled a system”.

Skill trust — proficiency × endorsements × usage-duration × assessment score.

Behavioural availability — recency, recruiter-response rate, open-to-work, notice period.

Trajectory — experience band fit + tenure stability (anti job-hopper).

Ranking Methodology

Retrieve → Score (Council of Nine) → Gate → Calibrate → Rank

$$\text{final} = \text{council_core} \times \text{integrity} \times \text{negative_screen} \times \text{availability} + \text{soft_nudges} \rightarrow \text{calibrated } 0-100$$

1 · Semantic Seer

dense JD ↔ profile similarity

2 · Name-Rectifier

title matches reality (anti-stuffer)

3 · Evidence Scout

"shipped a system" proof

4 · Mask-Piercer

verified skill-trust

5 · Path-Reader

experience band + tenure

6 · Terrain Master

product-vs-services + domain

7 · Neti-Neti Gate

negative screen (multiplier)

8 · Integrity Warden

honeypot / plausibility gate

9 · Availability Oracle

behavioural modifier

Each scorer maps a tested judgement principle (Confucius' "rectification of names", Sun Tzu, Kautilya's Arthashastra, Japanese honne/tatemaie) to a concrete, measurable feature.

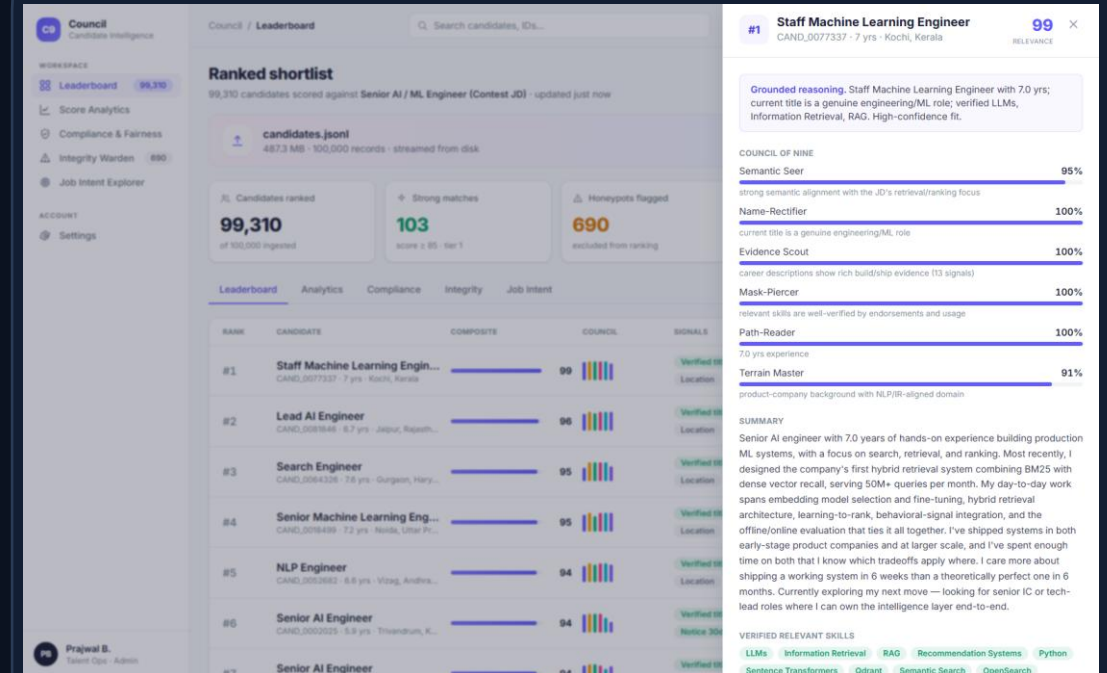
Explainability & Data Validation

How decisions are explained. Every candidate gets a 1–2 sentence grounded reason plus the full breakdown of all nine scores with a one-line rationale each.

No hallucinations. Reasoning is assembled only from fields that exist in the profile; skills are surfaced only if verified (endorsed / actually used) — the model can never cite a skill the candidate doesn't hold.

Honest concerns. Flaws are stated, not hidden (e.g. "strong fit, but 120-day notice"). Tone matches the rank.

Suspicious / low-quality profiles. The Integrity Warden runs logical-consistency checks — a role longer than the whole career, a skill used longer than the career, "expert" with 0 months — and excludes these honeypots from the ranking.



End-to-End Workflow

1. Ingest

Drag-drop a .json/.jsonl pool (any size) or use the default dataset.

2. Configure

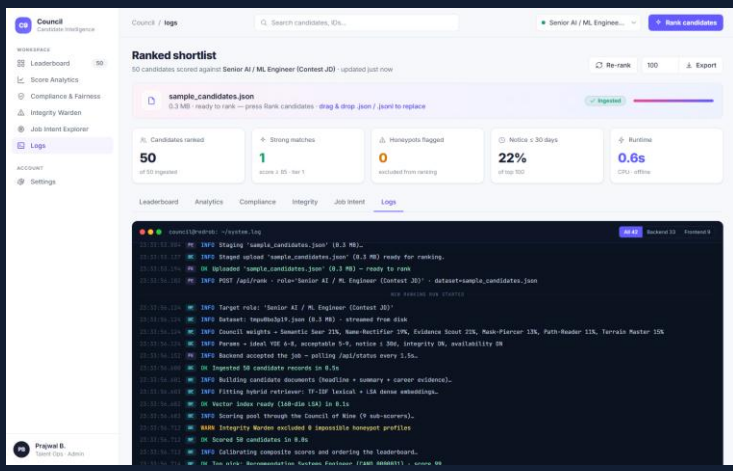
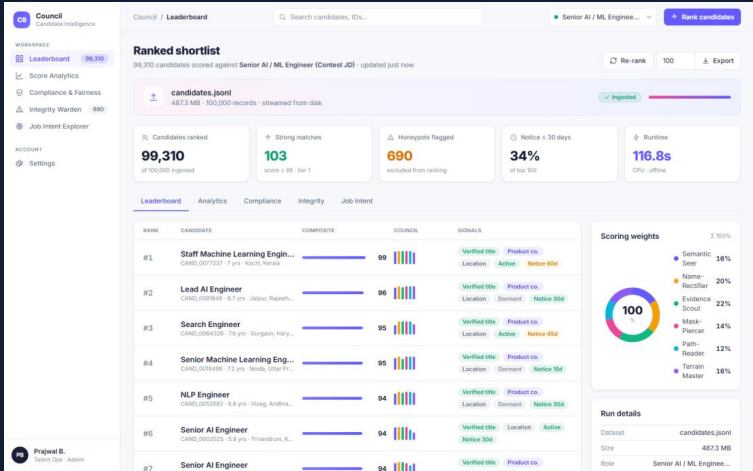
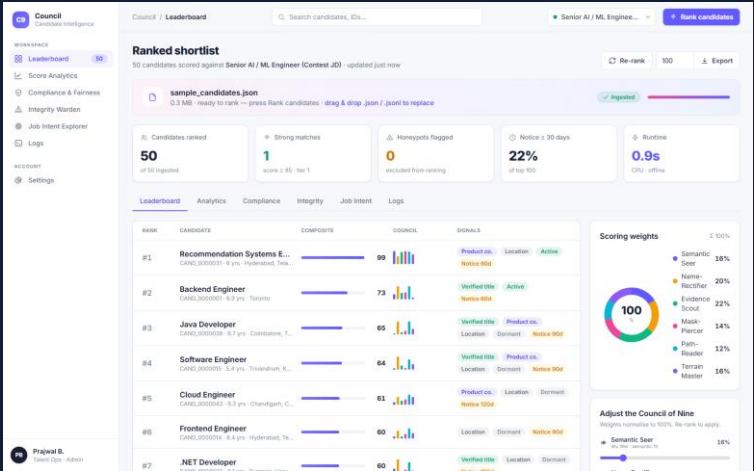
Pick the target role; tune the nine weights & parameters (YOE, notice, toggles).

3. Rank

Retrieve (TF-IDF + LSA) → score the whole pool → gate honeypots → calibrate.

4. Explore & Export

Leaderboard, candidate drawer, analytics, fairness, logs → export Excel.



System Architecture

Frontend · Next.js 14

- Stripe-style dashboard (Tailwind)
- Drag-drop upload, role + weights
- Leaderboard, drawer, analytics
- Live Logs console



API · FastAPI

- Background ranking job + polling
- /rank /status /leaderboard
- /candidate /analytics /compliance
- /honeypots /export /logs



Engine · Council of Nine (src/)

- load · features · retrieve
- council (9 scorers) · integrity
- fairness · reasoning · score
- CPU-only, offline, deterministic

Two-phase design: heavy index-building runs once; the online ranking step produces the shortlist in ~2 min on CPU with no network calls — meeting the challenge's ≤ 5 -min / 16 GB / offline constraint.

Results & Performance

100,000

candidates ranked

~90–135s

CPU · offline runtime

0.04–0.7%

honeypots flagged (DQ at 10%)

10 / 10

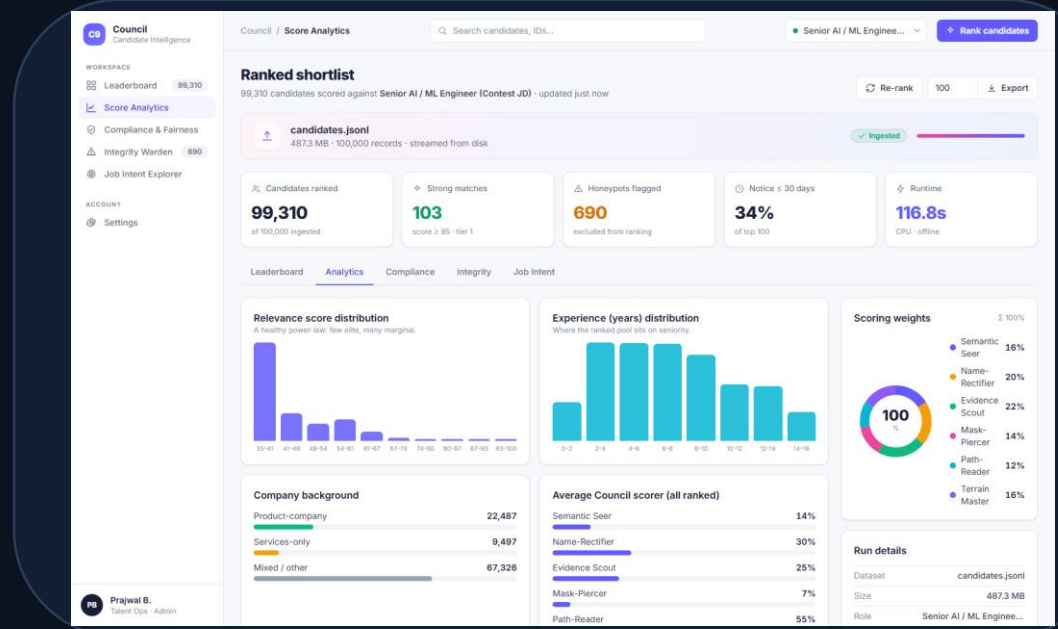
engine tests passing

Right people at the top. On the full pool, ranks 1–N are genuine Staff/Lead/Senior ML, AI, NLP & Search engineers — not keyword-stuffers.

Trap-resistant. Plain-language “hidden gems” beat keyword-perfect wrong-role profiles in our test suite.

Format-valid. Output passes the official `validate_submission.py` (“Submission is valid.”).

Budget-safe. ~2 min for 100K on CPU — ~2.5× under the 5-minute limit.



Technologies Used

Engine

- Python 3.13
- scikit-learn — TF-IDF + LSA
- NumPy / SciPy
- pandas + openpyxl (export)

Serving

- FastAPI + Uvicorn
- Background jobs + polling
- Pydantic models
- CORS / proxy

Frontend

- Next.js 14 + React 18
- Tailwind CSS
- SVG charts & donut
- Live logs console

Quality & sandbox

- Streamlit (hosted demo)
- Playwright (UI tests)
- Pluggable: Nomic/SPLADE/
- Qwen3 + ONNX (upgrade path)

Why this stack: a CPU-first, dependency-light core guarantees the engine reproduces inside the contest's offline 5-minute budget, while the architecture leaves a clean drop-in path to 2026 SOTA models (Nomic Embed, SPLADE v3, Qwen3-Reranker, ONNX) for production.

Submission Assets

GitHub repository

Full implementation — engine, API, web UI, tests, docs.

`github.com/<your-username>/AI-RECRUITING`

Demo video

Recorded walkthrough of the working system (this submission).

`Rainlight_Council_Demo.mp4`

Live sandbox

Hosted demo to run the ranker on a sample (Streamlit Cloud / HF Spaces).

`<your-sandbox-url>`

Ranked results

Top-100 shortlist in the required CSV format (validator-passed).

`submission.csv`

This deck

Approach, architecture & results (PDF + PPTX).

`Rainlight_Redrob_Submission.pdf`

Rainlight — The AI Recruiting Brain

We don't match keywords. We read careers, verify claims, weigh availability, and explain every decision — at India scale, on a laptop.

Team Rainlight · Varshith P